

Fun with Filters and Frequencies

Project 2 · Ariq Koh · CS180, Fall 2026 · convolution, edges and frequencies

This page covers Parts 1.1 to 1.3 and 2.1 to 2.4 so far, and the other parts will be added as they are finished. Click any image to open it at full size.

1.1. Convolution from scratch

Convolving an image with a kernel means flipping the kernel, sliding it over the image, and at every position multiplying it against the pixels underneath and summing the result. Here it is written in `numpy` only, first with four nested loops and then with two, on a zero-padded copy of the image. `scipy.signal.convolve2d` is used only as the reference to check against, and `cv2` only reads and saves images.

Padding

Both versions pad the image with zeros first so the kernel can be centered on the edge pixels.

Four loops

The outer two loops visit every output pixel, and the inner two loops walk over the kernel, accumulating one product at a time. The flip is the `[::-1, ::-1]` on the kernel, which reverses it along both axes.

```
def convol_img(image, kernel, mode="same"):
    # Perform convolution of the image with the given kernel (four for loop implementation)
    kernel_h, kernel_w = len(kernel), len(kernel[0])
    padded_image = pad_img(image, kernel, mode)
    flipped_kernel = np.array(kernel)[::-1, ::-1]
    height = padded_image.shape[0] - kernel_h + 1
    width = padded_image.shape[1] - kernel_w + 1
    output_image = np.zeros((height, width))

    for i in range(height):
        for j in range(width):
            sum_val = 0.0
            for ki in range(kernel_h):
                for kj in range(kernel_w):
                    sum_val += padded_image[i + ki][j + kj] * flipped_kernel[ki][kj]
```

```

        output_image[i][j] = sum_val

    return output_image

```

Two loops

The two inner loops are replaced by one array operation. The kernel-sized window of the padded image at the current pixel is sliced out, multiplied elementwise with the flipped kernel, and summed with `np.sum`.

```

def convol_img_2loops(image, kernel, mode="same"):
    # Perform convolution of the image with the given kernel (two for loop implementation)
    kernel_h, kernel_w = len(kernel), len(kernel[0])
    padded_image = pad_img(image, kernel, mode)
    flipped_kernel = np.array(kernel)[::-1, ::-1]
    height = padded_image.shape[0] - kernel_h + 1
    width = padded_image.shape[1] - kernel_w + 1
    output_image = np.zeros((height, width))

    for i in range(height):
        for j in range(width):
            output_image[i][j] = np.sum(padded_image[i:i + kernel_h, j:j + kernel_w] * flipped_kernel)

    return output_image

```

The kernels

The box filter averages a 9×9 neighborhood, so each entry is $1/81$ and the kernel sums to 1. D_x and D_y are the finite difference operators. Because convolution flips the kernel, convolving with `[1, -1]` gives $f(x) - f(x-1)$, the final value minus the initial one, so an edge that gets brighter to the right comes out positive.

```

Dx = [[1, -1]]
Dy = [[1], [-1]]

def box_kernel(size):
    # Generate a 2D box filter kernel (every entry is 1 / size^2 so it sums to 1)
    kernel = []
    for i in range(size):
        row = []
        for j in range(size):
            row.append(1 / size**2)
        kernel.append(row)
    return kernel

```

Checking against scipy

The reference call zero-fills the border the same way the padding above does. All three implementations run on the same image and kernel, in both padding modes, and the largest difference between their outputs is taken.

```
def convol_with_scipy(image, kernel, mode="same"):
    return scipy.signal.convolve2d(image, kernel, mode=mode, boundary="fill", fillvalue=0)

# in part1.py, for each of the box filter, Dx and Dy, and for mode in ["same", "full"]:
cimg, t_four = timed(convol_img, img, kernel, mode)
c2img, t_two = timed(convol_img_2loops, img, kernel, mode)
scimg, t_scipy = timed(convol_with_scipy, img, kernel, mode)
max_diff = max(np.abs(cimg - scimg).max(), np.abs(c2img - scimg).max())
```

Results

The photo is read as grayscale with `cv2.imread(..., cv2.IMREAD_GRAYSCALE)` and convolved with the 9×9 box filter, Dx and Dy, using "same" padding. The four-loop, two-loop and `scipy` outputs are identical, and the saved images below come from the four-loop version.



Figure 1. The input, read as grayscale.



Figure 2. Convolved with the 9×9 box filter. It blurs the image, and the border is darker where the kernel overlaps the zero padding.



Figure 3. Convolved with Dx. Vertical edges light up: the tree line, the paddle and the paddler's legs. Absolute value, scaled so the largest response is 255.



Figure 4. Convolved with Dy. Horizontal edges light up: the ridge line, the shoreline and the water ripples. Absolute value, scaled so the largest response is 255.

Runtime

Each implementation was timed once on the full 1280×960 image.

KERNEL	MODE	FOUR LOOPS	TWO LOOPS	SCIPY
9×9 box	same	26.20 s	2.58 s	0.084 s
9×9 box	full	26.25 s	2.49 s	0.084 s
Dx	same	0.93 s	2.44 s	0.009 s
Dx	full	0.95 s	2.46 s	0.008 s
Dy	same	1.00 s	2.51 s	0.012 s
Dy	full	0.98 s	2.47 s	0.011 s

The four-loop version does one Python-level multiply-add per kernel entry per pixel, about 100 million of them for the 9×9 box filter, so its time grows with the kernel: 26 s for the box filter against about 1 s for the two-entry Dx and Dy. The two-loop version makes a handful of NumPy calls per pixel instead, so its time barely depends on the kernel, about 2.5 s for all three. That makes it roughly 10 times faster for the box filter, but about 2.5 times slower for Dx and Dy, where the four-loop

version only has two products to add and NumPy's per-call overhead costs more than it saves.

`scipy` is much faster than either: 0.084 s for the box filter, about 30 times faster than the two-loop version and 300 times faster than the four-loop one, and about 0.01 s for Dx and Dy, because its loops run in compiled code instead of Python.

Boundary handling

Both implementations pad with zeros, which is the fill `convolve2d` uses by default, so the outputs match `scipy` up to floating-point rounding (a largest difference of $6.3e-13$ for the box filter and 0 for Dx and Dy). Zero padding only affects the outermost pixels: the box filter darkens the outer 4 pixels because it averages in zeros, and Dx and Dy pick up a one-pixel edge along the left and top border respectively. `scipy` can also reflect or wrap at the border, but only zero fill is implemented here.

1.2. Finite difference operator

The cameraman image is convolved with the finite difference operators from Part 1.1, using the four-loop convolution with "same" padding. Dx gives the partial derivative I_x of the image I in x and Dy gives the partial derivative I_y in y. The gradient magnitude, $\sqrt{I_x^2 + I_y^2}$, combines the two into a single edge-strength image that doesn't depend on which way the edge points.



Figure 5. The input, cropped and read as grayscale.



Figure 6. Gradient magnitude. The outline, camera, tripod and horizon show up together, along with faint speckle from the grass. Scaled so the largest value is 255.



Figure 7. I_x , from D_x . Vertical edges light up: the tripod legs, the buildings on the horizon and the outline of the coat. Absolute value, scaled so the largest response is 255.



Figure 8. I_y , from D_y . Horizontal edges light up: the camera, the shoulder and the line of the horizon. Absolute value, scaled so the largest response is 255.

Choosing a threshold

A binary edge image keeps the pixels of the gradient magnitude above a threshold. Figure 9 shows four choices.



Figure 9a. Threshold 15 (12.6% of pixels). Too low: the grass and specks in the sky pass.



Figure 9b. Threshold 30 (6.4%). The outline and tripod are clean, but the grass still speckles the bottom.

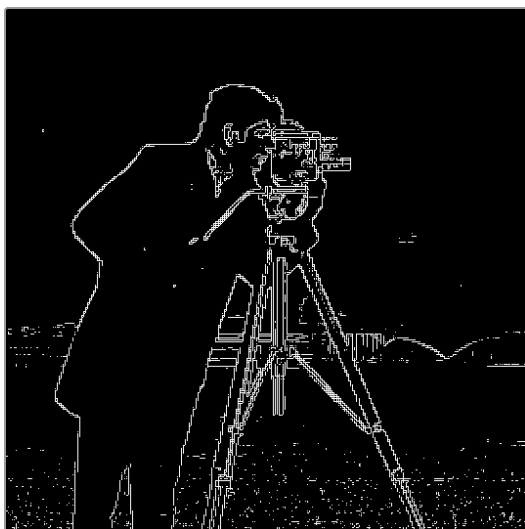


Figure 9c. Threshold 45 (3.8%), the one used.



Figure 9d. Threshold 80 (1.7%). Too high: the camera detail and the horizon break up or vanish.

The threshold trades noise against real edges. At 15 the fine texture of the grass and a few specks in the sky are strong enough to pass, so noise crowds the real edges. At 80 the noise is gone, but so is a lot of true structure: the details of the camera and the skyline at the horizon are broken up or missing. Threshold 45 works best. It removes most of the grass texture while keeping the cameraman's outline, the camera, the tripod legs and most of the horizon.

1.3. Derivative of Gaussian (DoG) filter

Finite differences amplify noise, which is why the edge maps in Part 1.2 pick up the texture of the grass. Blurring first suppresses it. The Gaussian here is 13×13 with $\sigma = 2$, built as the outer product of the 1D kernel from `cv2.getGaussianKernel` with itself,

$$G = gg^T$$

blurred the image with it, and repeated the Part 1.2 procedure on the blurred result.



Figure 10a. Without blur (the input from Figure 5).



Figure 10b. Blurred with the 13x13 Gaussian, $\sigma = 2$.

DoG filters

Convolution is associative, so blurring the image and then taking a finite difference is the same as convolving the image once with the Gaussian already convolved with the difference operator. Convolving the Gaussian with D_x and with D_y , using the convolution from Part 1.1 with "full" padding, gives the two derivative of Gaussian (DoG) filters:

$$\text{DoG}_x = G * D_x \quad \text{DoG}_y = G * D_y$$

$$I * \text{DoG}_x = (I * G) * D_x$$

Here I is the input image, G is the Gaussian, D_x and D_y are the difference operators from Part 1.1, and $*$ is convolution. The filters come out 13x14 and 14x13, one row or column larger than the Gaussian because D_x and D_y have two entries.

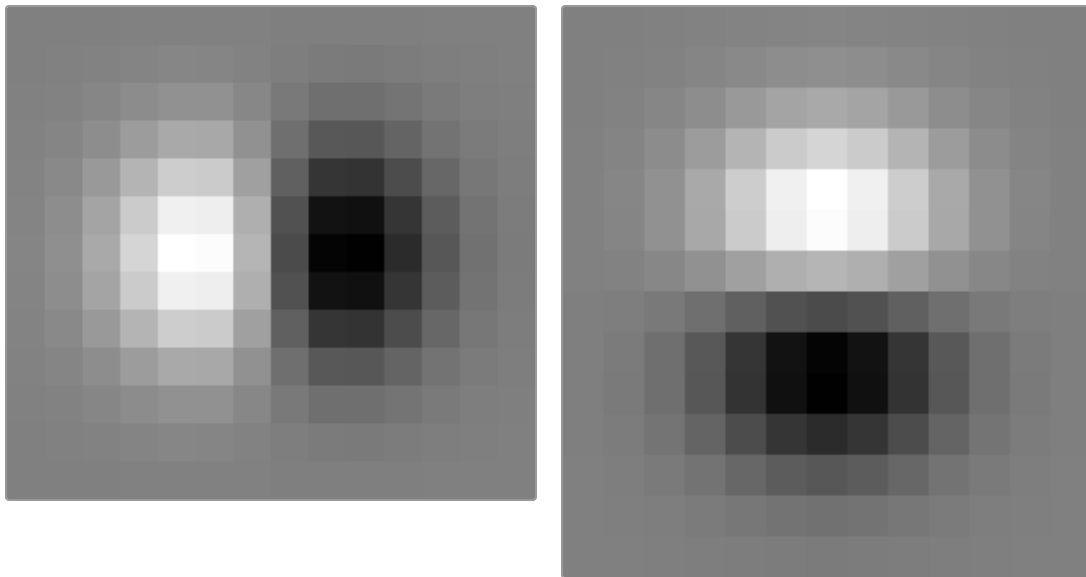


Figure 11. The DoG filters, DoG_x (left, 13×14) and DoG_y (right, 14×13), enlarged $20\times$. Mid gray is zero, lighter is positive and darker is negative.

Blurred against unblurred

Figure 12 compares the gradient magnitude and the binarized edges without blur (Part 1.2) and with it. The blurred gradients are much weaker, with a maximum of 39.6 against 221, so they use a lower threshold of 15, chosen the same way as before, instead of 45. The blur and the derivatives here mirror the image at the border instead of zero padding it, so the blur doesn't darken the edges and no fake frame appears.



Figure 12a. Gradient magnitude, no blur (Part 1.2).



Figure 12b. Gradient magnitude, blurred first.



Figure 12c. Binarized edges, no blur, threshold 45.

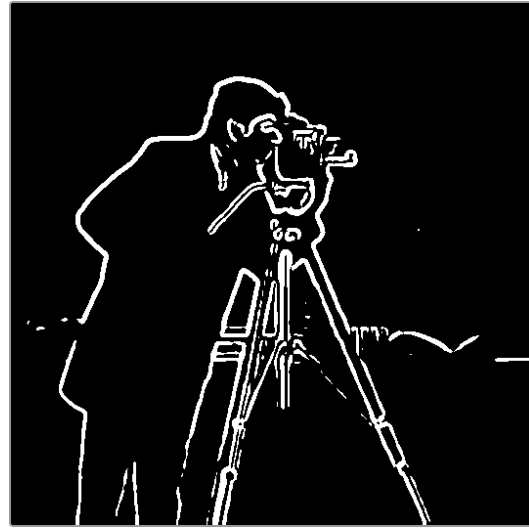


Figure 12d. Binarized edges, blurred first, threshold 15.

DoG results

Applying the DoG filters directly to the original image, in one convolution each, gives the same result as blurring first. The largest difference from the two-step version is $2.5e-13$ for I_x , $2.1e-13$ for I_y and $2.7e-13$ for the magnitude, which is floating-point rounding, and none of the 285,156 pixels differ in the binarized edges.



Figure 13a. Gradient magnitude from the DoG filters, matching Figure 12b.

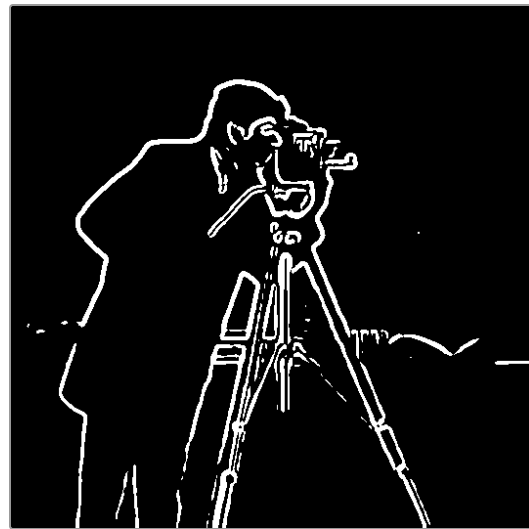


Figure 13b. Binarized edges from the DoG filters, threshold 15, matching Figure 12d.

What differences do you see?

Blurring removes the noise. Without it, the grass texture and the specks in the sky pass a low threshold, and it takes a high one (45) to get rid of them, which also thins out real edges. With the blur the grass produces almost no false edges, so a much lower threshold keeps the outline, the camera, the tripod and much of the horizon. The cost is sharpness. The edges come out several pixels

wide instead of one or two, and small details such as the parts of the camera are smoothed into fewer, rounder shapes.

2.1. Image sharpening

Unsharp masking sharpens an image by boosting its high frequencies. A Gaussian blur G keeps only the low frequencies, so subtracting the blurred image from the original leaves the high frequencies: the edges and fine texture that the blur removed. Adding some of that back sharpens the image, and α sets how much:

$$I_{\text{sharp}} = I + \alpha(I - I * G)$$

$$= (1 + \alpha)I - \alpha(I * G) = I * ((1 + \alpha)\delta - \alpha G)$$

Here δ is the unit impulse, a kernel with a 1 in the center and 0 everywhere else, so that $I * \delta = I$. Convolution is linear, so blurring, subtracting and adding back collapse into one convolution with the kernel $(1 + \alpha)\delta - \alpha G$. That kernel sums to 1, so the overall brightness doesn't change. The kernel is built once and the image is convolved with it, mirroring the border. It matches blurring, subtracting and adding back to within 7e-13.

```
def unsharp_kernel(size, sigma, alpha):
    # Unsharp mask as a single kernel. image + alpha * (image - image * G) is linear in the image, so it
    # equals image * ((1 + alpha) * impulse - alpha * G), where the impulse is a 1 at the center of the
    kernel = -alpha * gaussian_kernel_cv(size, sigma)
    kernel[size // 2, size // 2] += 1 + alpha
    return kernel

def unsharp(image, kernel, sigma, alpha=1.0):
    # Apply unsharp masking with a single convolution, alpha is how much of the high frequencies to add
    return convol_symm_with_scipy(image, unsharp_kernel(kernel, sigma, alpha))
```

taj.jpg

This example uses a 5×5 Gaussian with $\sigma = 1$ and $\alpha = 1$. Figure 14c is the high-frequency component, scaled up 4× and shifted to mid gray so that both the positive and the negative detail show up.



Figure 14a. The original.



Figure 14b. Blurred with the 5x5 Gaussian, $\sigma = 1$.



Figure 14c. The high frequencies, $I - I * G$, scaled 4x around mid gray.



Figure 14d. Sharpened, $\alpha = 1$.

Sharpening amount

Figure 15 varies α with the same Gaussian. At 0.5 the change is mild, at 1 and 2 the edges are clearly crisper, and at 4 the image starts to look overdone, with halos along the edges of the trees and a gritty texture on the marble.



Figure 15a. $\alpha = 0.5$.



Figure 15b. $\alpha = 1$.



Figure 15c. $\alpha = 2$.

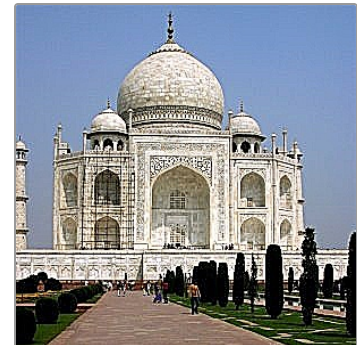


Figure 15d. $\alpha = 4$.

Blur, then sharpen

To see how much sharpening can win back, `taj.jpg` is blurred with a stronger 11x11 Gaussian ($\sigma = 1.5$), sharpened with the same Gaussian, and compared with the original by eye. $\alpha = 2$ brings the edges back without visible halos.



Figure 16a. The original.



Figure 16b. Blurred, $\sigma = 1.5$.



Figure 16c. The blurred image sharpened again, $\alpha = 2$.

Sharpening gets back some of what the blur took: the edges of the arches and the trees look crisper again, and the result is clearly closer to the original than the blurred image is. It doesn't restore the original, though. Detail that the blur wiped out completely, like the fine carving on the dome, can't come back, because sharpening only amplifies what is left. Pushing α higher only adds halos and noise, as in Figure 15d.

Other images

The same procedure is applied to two other photos, a plate of duck and a shot of fish in an aquarium, both slightly out of focus.



Figure 17a. The original, shrunk to 1000 px.



Figure 17b. Blurred, $\sigma = 2$.

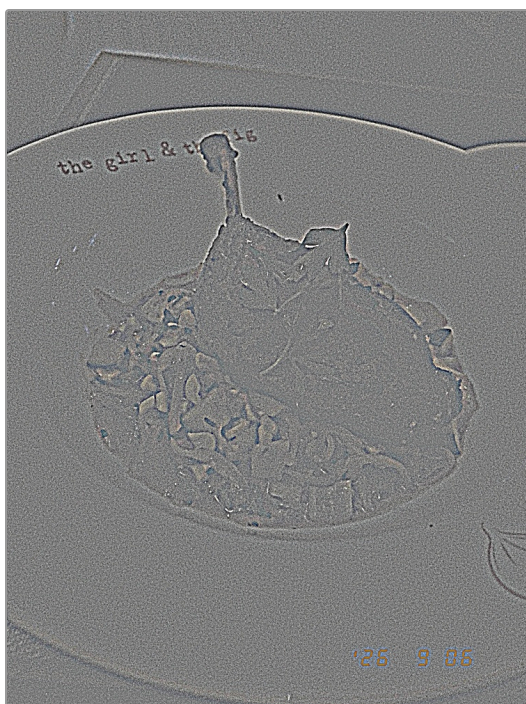


Figure 17c. The high frequencies, scaled 4x around mid gray.



Figure 17d. Sharpened, $\alpha = 1$.



Figure 18a. The original, shrunk to 1000 px.



Figure 18b. Sharpened, $\alpha = 1$.

The lettering on the plate and the veins of the parsley come out crisper, and the edges and fins of the fish are firmer. Both photos have visible film grain, and sharpening amplifies it along with the detail (it is clearly visible in Figure 17c), which is the main cost here.

2.2. Hybrid images

A hybrid image (Oliva, Torralba and Schyns, SIGGRAPH 2006) mixes the high frequencies of one image with the low frequencies of another. Up close the eye picks up the fine detail, so you see the first image. From far away that detail is too small to see, and only the low frequencies are left, so you see the second. The high frequencies of I_1 are the image minus its Gaussian blur, which is the same as convolving it with an impulse minus a Gaussian. They are scaled by a gain k and added to the blur of I_2 :

$$\begin{aligned} H &= k(I_1 - I_1 * G_{\sigma_1}) + I_2 * G_{\sigma_2} \\ &= I_1 * k(\delta - G_{\sigma_1}) + I_2 * G_{\sigma_2} \end{aligned}$$

Here G_σ is a Gaussian with standard deviation σ pixels, δ is the unit impulse, and σ_1 and σ_2 set the two cutoff frequencies. The gain k sets how strong the first image is next to the second: at $k = 1$ the high frequencies are used as they are, and a pair that needs the first or the second image to stand out more gets a larger or smaller value. The blur is done as two 1D passes, because the σ needed here would make a single 2D kernel slow.

```
def gaussian_blur(image, sigma):
    # Blur with a separable Gaussian (two 1D passes), much faster than one 2D kernel when sigma is large
    size = 2 * int(np.ceil(3 * sigma)) + 1
    kernel_1d = cv2.getGaussianKernel(size, sigma)
    return convolve_symm_with_scipy(convolve_symm_with_scipy(image, kernel_1d), kernel_1d.T)

def hybrid_image(im1, im2, sigma1, sigma2, gain=1.0):
    # High frequencies of im1 (the image minus its blur, times a gain that sets how strong they are next
    # low frequencies) plus the low frequencies of im2 (its blur)
    high = gain * (im1 - gaussian_blur(im1, sigma1))
    low = gaussian_blur(im2, sigma2)
    return high + low
```

Aligning the images

The two images have to line up, or the low-frequency image reads as a different thing behind the detail. Two matching points are picked on each image, and the course starter code translates, scales and rotates the first image so that the two points land on top of each other. The eyes are used for the person and the cat and for the two dolls, and two points along the railing for the gulls. Both images of every pair are converted to grayscale first, so that the hybrid depends only on brightness and the two frequency bands don't fight over color.

Dessert

Figures 19 to 21 walk through every step of the dessert hybrid, the favorite of the set. The meringue tart supplies the high frequencies and the red pastry the low ones. The two are aligned on the top of each dessert and the bottom of its gold base.



Figure 19a. Meringue tart, the high-frequency image.



Figure 19b. Red pastry, the low-frequency image.

Figure 20 shows each step. The two aligned images (20a and 20b) are cropped to 576×862 . The tart's image has its blur removed ($\sigma_1 = 7$), leaving only the edges (20c, shifted to mid gray), and the pastry's is blurred ($\sigma_2 = 16$, 20d). Adding them gives the hybrid (20e).



Figure 20a. Tart, aligned.



Figure 20b. Pastry, aligned.

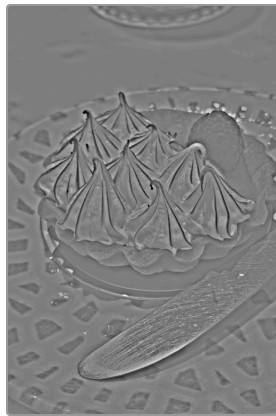


Figure 20c. Tart, high-pass.

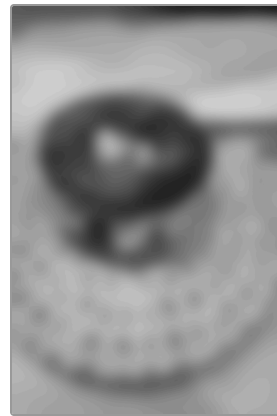
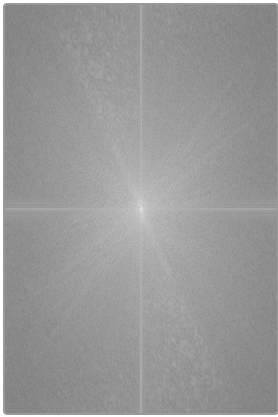
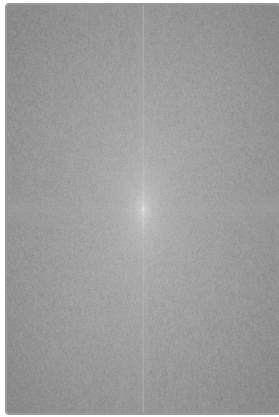
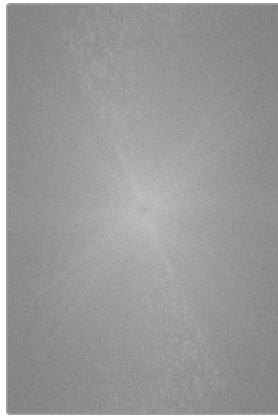
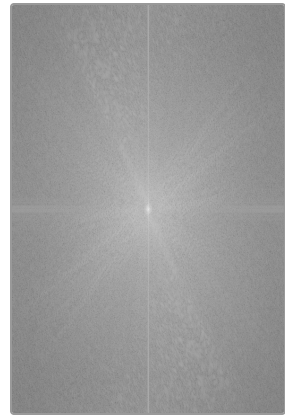


Figure 20d. Pastry, low-pass.



Figure 20e. The hybrid.

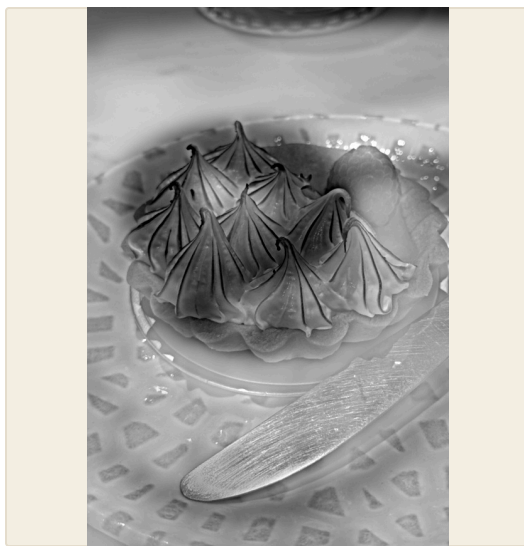
Figure 21 shows the log magnitude of the 2D Fourier transform of each image, all on the same brightness scale, with the lowest frequencies in the center. The two inputs have energy at all frequencies, brightest at the center and fading outwards. The bright cross through the center of each spectrum comes from the image borders.

**Figure 21a.** Tart.**Figure 21b.** Pastry.**Figure 21c.** Tart, high-pass.**Figure 21d.** Pastry, low-pass.**Figure 21e.** The hybrid.

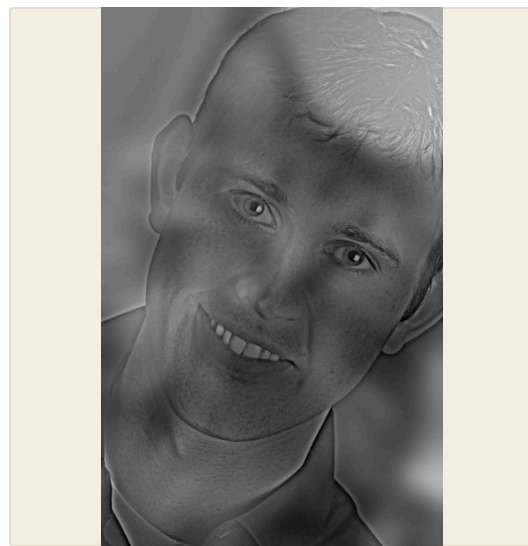
Viewing it

Each hybrid in Figure 22 has its own slider that moves it away from you. Close up you should see the high-frequency image, and as it gets smaller the low-frequency one takes over.

(In print, each slider below is frozen at its starting, close-up position. View the live page for the working slider.)



Close  Far
Close up

Figure 22a. Dessert.

Close  Far
Close up

Figure 22b. Derek and Nutmeg.

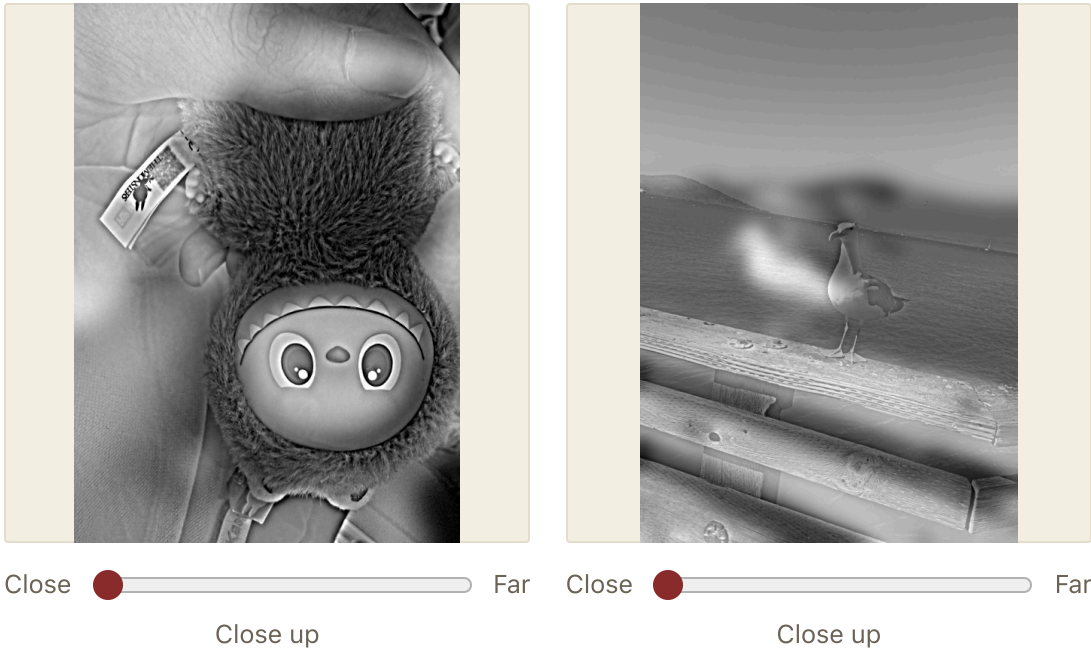


Figure 22c. Labubu.

Figure 22d. Bird.

Cutoff frequencies

The two cutoffs were found by trying many combinations for each pair and judging the result twice, at the size the slider starts at and again shrunk to a fraction of that. The first image should win up close and the second from far away. A larger σ_1 lets more of the first image through, and a larger σ_2 blurs the second more. The gain k shifts the balance between the two: a larger value strengthens the first image and a smaller one lets the second stand out.

The settings were tuned again after converting to grayscale, since the far view then has to work on brightness alone. The dessert and Derek and Nutmeg use $k = 1$. The labubu uses $k = 2$, so that the purple doll's fur and face lines stand out, and the gull uses $k = 0.65$, so that the soft white shape of the wings-spread gull is easier to see from far away.

HYBRID	HIGH FREQUENCIES FROM	LOW FREQUENCIES FROM	HIGH-PASS	LOW-PASS	GAIN K
Dessert	Meringue tart	Red pastry	$\sigma = 7$ (0.023)	$\sigma = 16$ (0.010)	1
Derek and Nutmeg	Derek	Nutmeg	$\sigma = 4$ (0.040)	$\sigma = 12$ (0.013)	1
Labubu	Purple doll	Brown doll	$\sigma = 5$ (0.032)	$\sigma = 18$ (0.009)	2
Bird	Standing gull	Gull with wings spread	$\sigma = 5$ (0.032)	$\sigma = 12$ (0.013)	0.65

σ is in pixels, and the cutoff in cycles per pixel is in brackets.

More hybrids

These are the other three hybrids. For each one the two inputs are shown in color, in the order high frequencies, then low frequencies, followed by the hybrid, which is made from their grayscale versions. Together with the dessert they morph one dessert into another, a person into a cat and one doll into another, and change the pose of a gull.



Figure 23a. Derek, high frequencies.



Figure 23b. Nutmeg, low frequencies.



Figure 23c. The hybrid, aligned on the eyes.

Derek supplies the high frequencies and Nutmeg the low ones, and they are aligned on their eyes. Up close you see Derek's face drawn in fine lines, and from far away it turns into the cat.



Figure 24a. Purple doll, high frequencies.



Figure 24b. Brown doll, low frequencies.

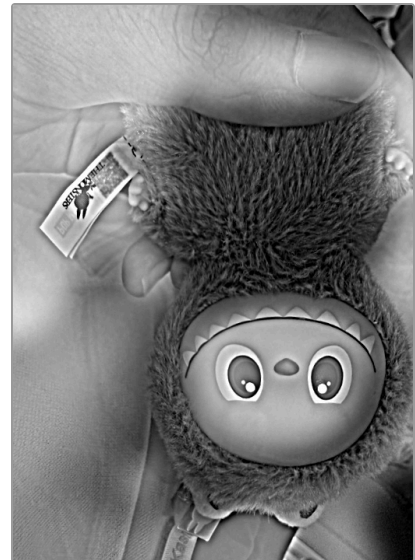


Figure 24c. The hybrid, aligned on the eyes.

The two dolls are held in almost the same pose, so aligning on their eyes lines up the faces. Up close you see the purple doll's fur and face detail, and from far away it turns into the darker, brown one. Because the dolls look so much alike, the purple doll's high frequencies get double the gain, so that its fur and the outlines of its face stand out.



Figure 25a. Standing gull, high frequencies.



Figure 25b. Gull with wings spread, low frequencies.

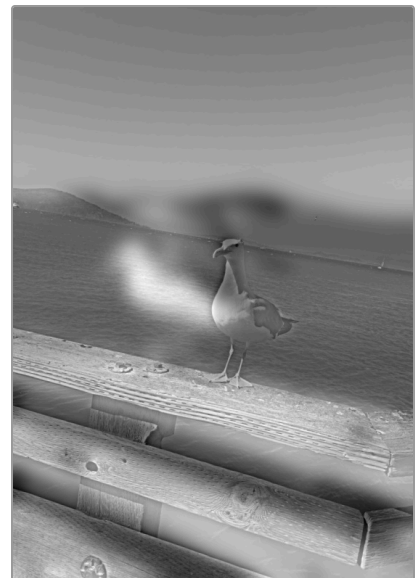


Figure 25c. The hybrid, aligned on the railing.

This one changes a gull's pose instead of swapping objects. The two photos are aligned on the railing, using two points on the far edge of its top board in each, which turns the first photo about 7° and shrinks the second to about 79%. The gulls are in different places along the railing, so they don't land on top of each other: the standing gull is drawn in fine outlines on the right, and the wings-spread one is a soft white shape on the left. The standing gull's high frequencies have a gain of 0.65 so that the white shape is easier to see, and from far away it is what is left.

2.3. Gaussian and Laplacian stacks

A Gaussian stack blurs an image with a wider Gaussian at each level. Unlike a pyramid it never subsamples, so every level keeps the full size of the image and all the levels sit in one array. Level 0 is the image itself, and each level after it blurs the original with a Gaussian twice as wide as the one before. A Laplacian stack takes the difference between neighboring Gaussian levels, so each level holds one band of frequencies, and it keeps the blurriest Gaussian level at the end. The levels add back up to the image:

$$L_i = G_i - G_{i+1} \quad L_N = G_N$$

$$I = \sum_{i=0}^N L_i$$

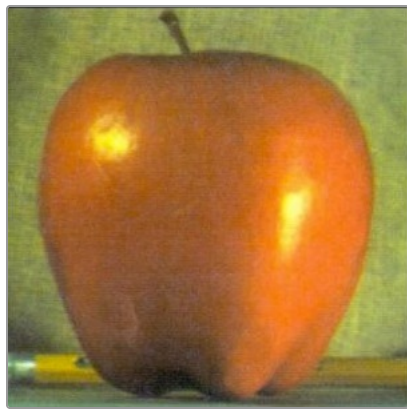
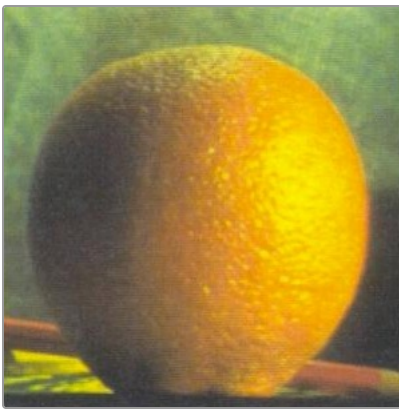
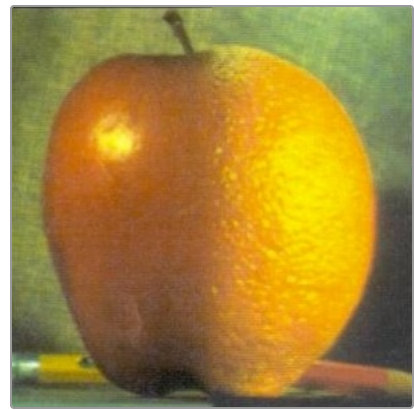
Neither stack uses `cv2.pyrDown` or `skimage.transform.pyramid_gaussian`. Each blurred level uses the separable Gaussian blur from the hybrid images, and there are five levels in all. The Laplacian levels add back to each image to within about 1e-16.

```
def gaussian_stack(image, levels, sigma=4):
    # Gaussian stack: level 0 is the image, and every level after it blurs the image with a Gaussian twice
    # the level before. Nothing is subsampled, so all levels keep the size of the image
    stack = [image]
    for i in range(1, levels):
        stack.append(gaussian_blur(image, sigma * 2 ** (i - 1)))
    return np.array(stack)

def laplacian_stack(gaussian):
    # Laplacian stack: the difference between neighboring Gaussian levels, with the blurriest Gaussian level
    # at the end, so the levels add back up to the image
    stack = []
    for i in range(len(gaussian) - 1):
        stack.append(gaussian[i] - gaussian[i + 1])
    stack.append(gaussian[-1])
    return np.array(stack)
```

Orapple

The course's apple and orange images (300×300) come first, with $\sigma = 2, 4, 8$ and 16 for the blurred levels. Figure 26 shows the two images and the blend built from their stacks.

**Figure 26a.** Apple.**Figure 26b.** Orange.**Figure 26c.** The blend.

Figures 27 and 28 show the Gaussian stack on top and the Laplacian stack below it for each image. Level 0 of the Laplacian stack holds only the finest detail, such as the speckle of the skin and the outline of the fruit, and each level after it holds coarser structure. Level 4 is the blurry remainder that carries the overall color.

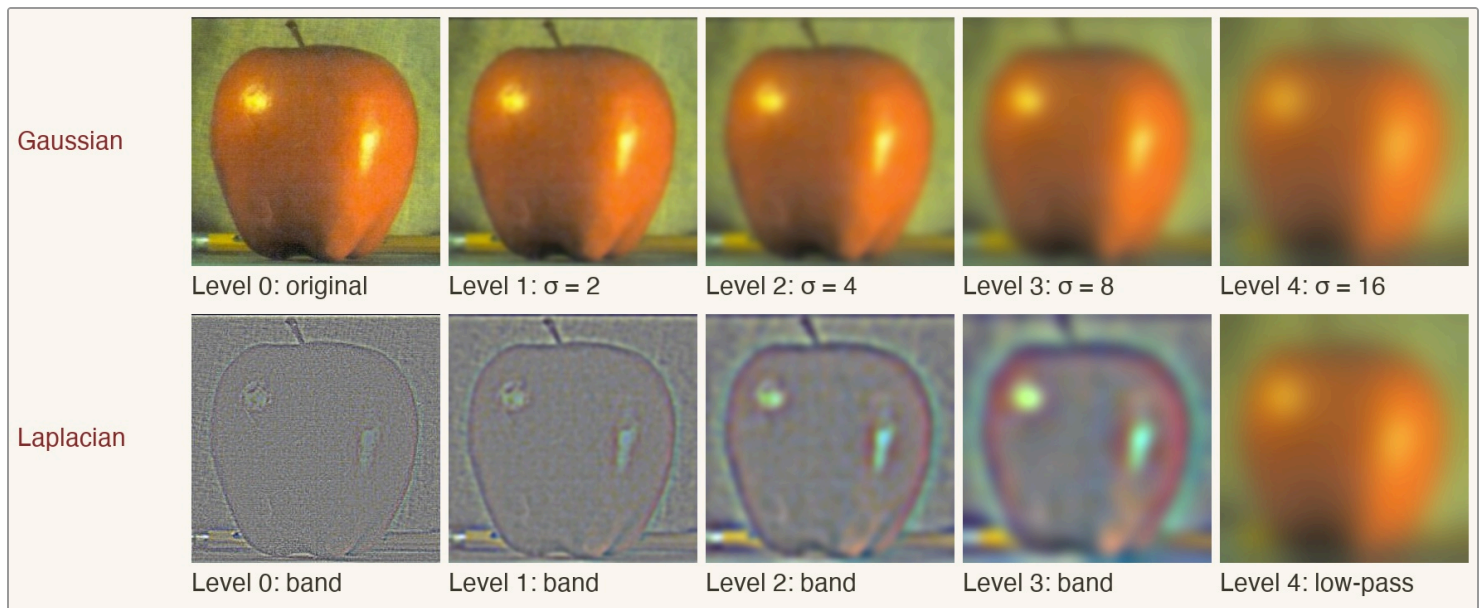


Figure 27. Gaussian stack (top) and Laplacian stack (bottom) of the apple. The band levels are scaled up 3× and shifted to mid gray.

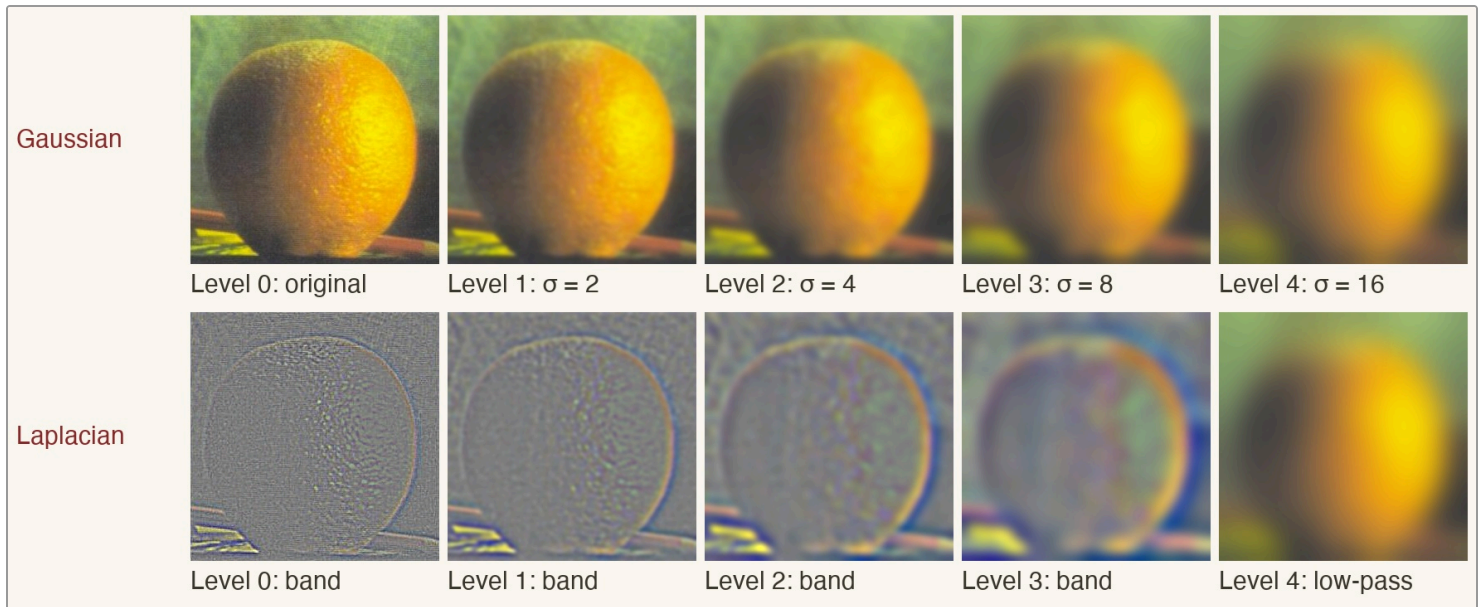


Figure 28. The same for the orange.

Figure 29 recreates Szeliski's Figure 3.42. The mask is 1 on the left half and 0 on the right, and its own Gaussian stack gives the weight at each level, so the seam is sharp at level 0 and gets wider at every coarser level. The rows show levels 0, 2 and 4. The first two columns are the apple and the orange weighted by the mask, and the third column is their sum. The last row adds up all the levels, and the sum in the last column is the blended image.

```
def blend_stacks(laplacian_a, laplacian_b, mask_gaussian):
    # Blend two Laplacian stacks level by level, weighting level i of the first by level i of the mask's
    # and level i of the second by what is left. Returns every blended level, and their sum is the blend
    mask = mask_gaussian[..., np.newaxis]
    return mask * laplacian_a + (1 - mask) * laplacian_b
```

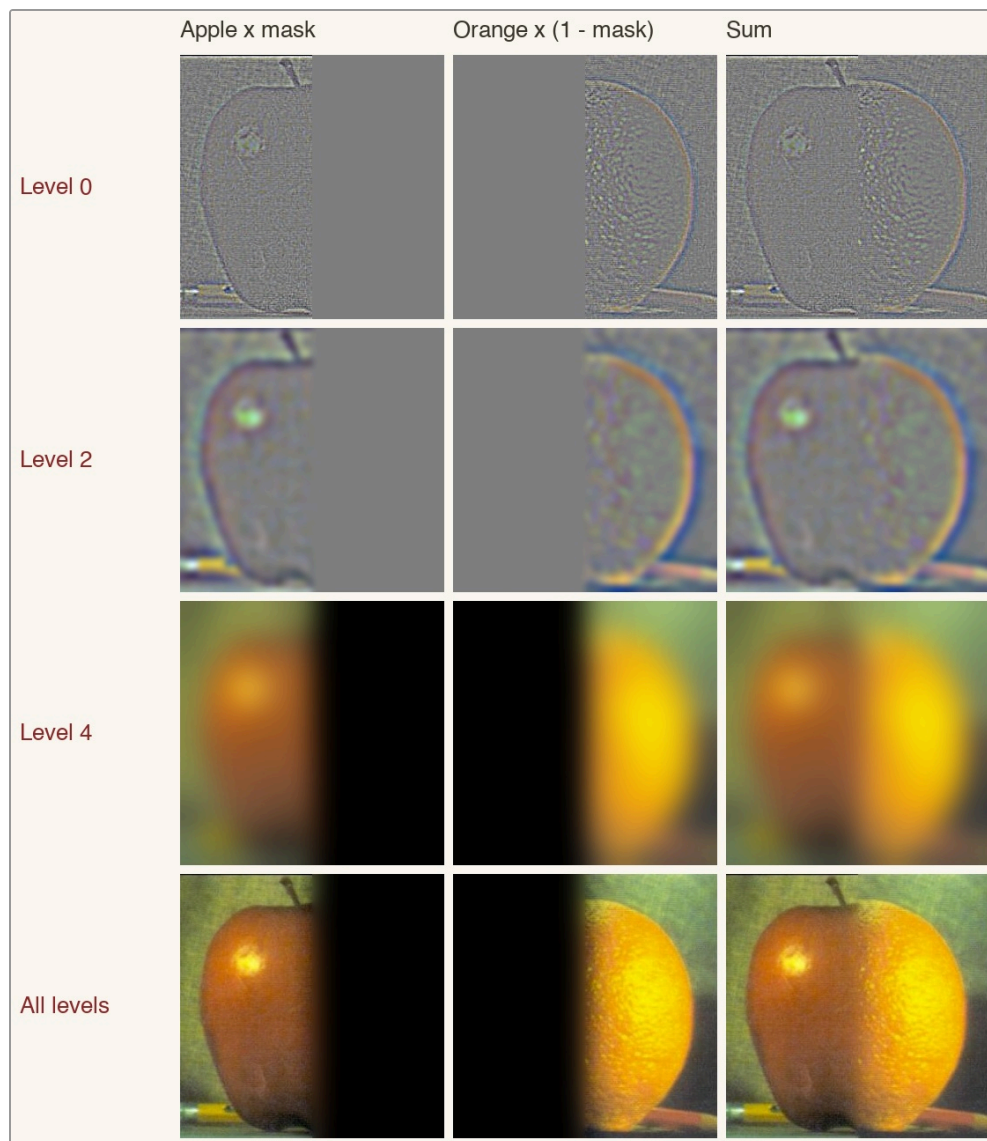


Figure 29. The apple and orange weighted by the mask at three levels, and the total over all levels.

Plush toys

The same procedure runs on two plush toys, Pikachu and Charizard (1200×1200), with $\sigma = 4, 8, 16$ and 32 for the blurred levels, since the images are four times larger. Charizard is scaled up 5% and shifted down to line up with Pikachu at the seam, since the two figures otherwise sat at different heights in their photos.



Figure 30a. Pikachu.



Figure 30b. Charizard.



Figure 30c. The blend.



Figure 31. Gaussian stack (top) and Laplacian stack (bottom) of Pikachu.

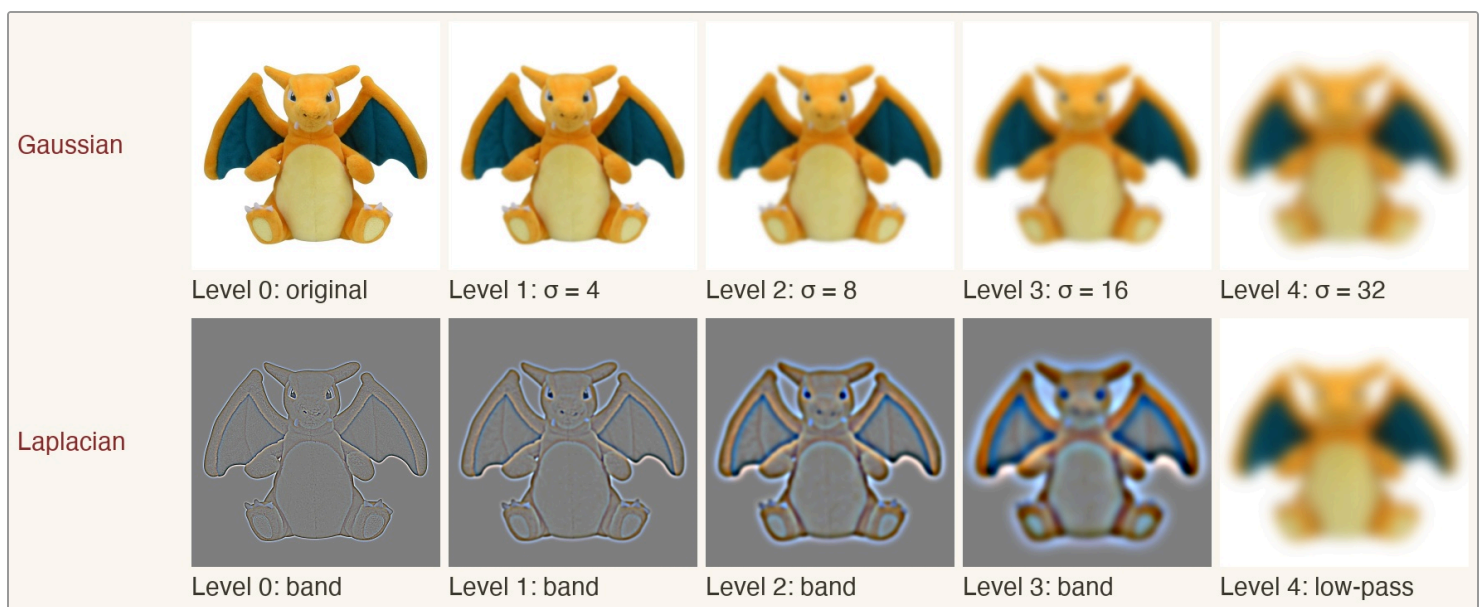


Figure 32. The same for Charizard.

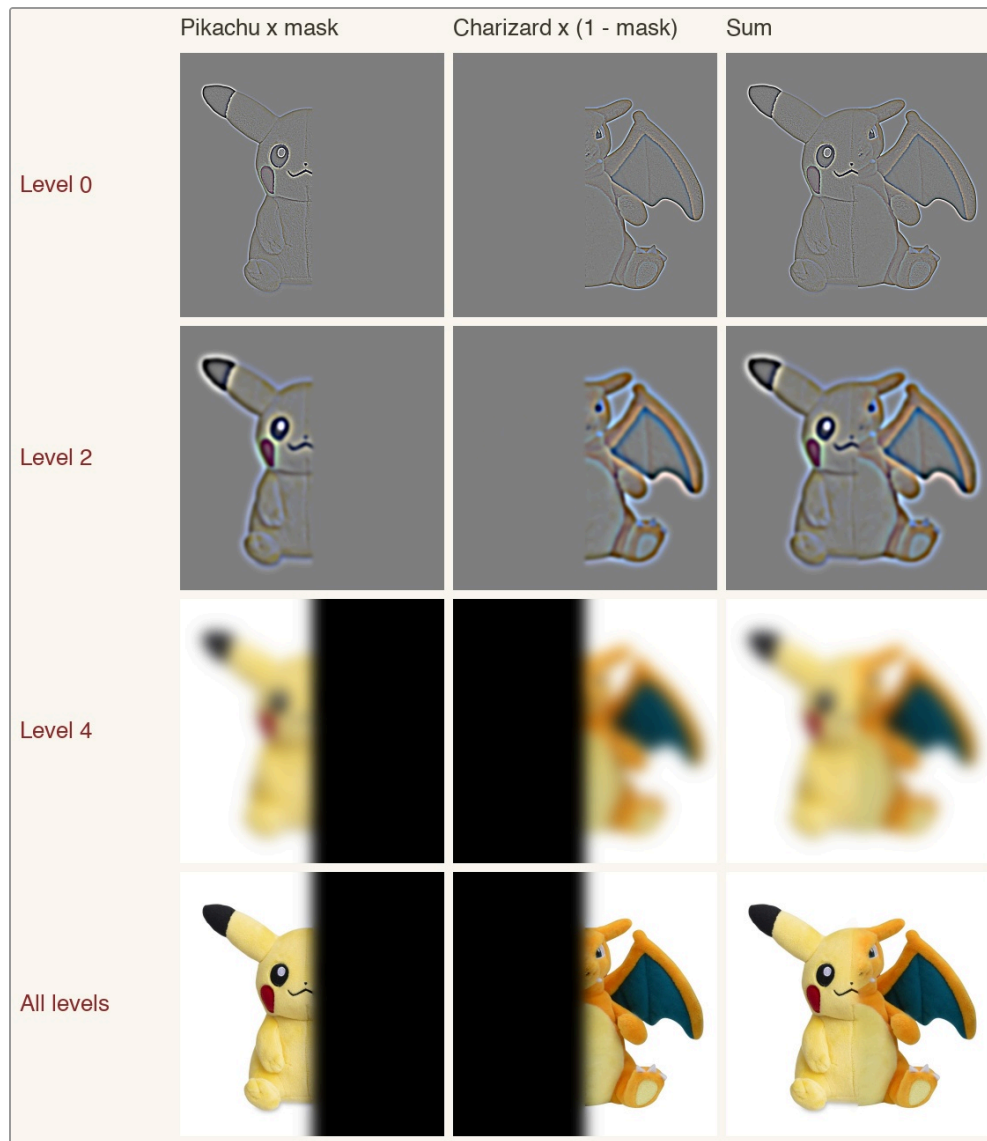


Figure 33. Pikachu and Charizard weighted by the mask at three levels, and the total over all levels.

Pikachu's left half and Charizard's right half meet along a smooth seam, and each half keeps its own fine fur detail. The white backgrounds blend without a visible edge, since both images are white on either side of the seam.

2.4. Multiresolution blending

Multiresolution blending combines two images by blending their Laplacian stacks level by level, weighted at each level by a mask's own Gaussian stack (`blend_stacks`, from Part 2.3). Coarse levels blend over a wide area and fine levels over a narrow one, since the mask's own blur widens with the level, which is what keeps a sharp object sharp while its low frequencies fade smoothly into the background. A mask that is a step function gives a straight seam, and the Orapple and the plush toys in Part 2.3 (Figures 26 and 30) already cover that case. A mask that follows an object's silhouette

instead of a straight line blends the object into a scene without a visible edge, which is what this section adds.

Irregular / creative mask

The two source images are a "this is fine" panel and a still of Spongebob relaxing in his living room. Spongebob is cut out of his photo with `cv2.grabCut`, seeded with a rectangle around his head and torso that excludes the armchair behind him and the floaty he sits on. The raw GrabCut mask has small holes where dark details (his belt, his tie) get classified as background, so it is filled to its external contour, giving a solid silhouette instead of a sticker with pinholes in it.

```
def segment_grabcut(image, rect, iterations=10):
    # Cut a rough foreground object out of a BGR uint8 image with GrabCut, seeded with a rectangle around
    # up with morphological open/close and filled to its external contour, so interior dark details (but
    # don't punch holes in the mask. Returns a float64 mask of 1s and 0s, the same size as the image
    gc_mask = np.zeros(image.shape[:2], np.uint8)
    bgd_model, fgd_model = np.zeros((1, 65), np.float64), np.zeros((1, 65), np.float64)
    cv2.grabCut(image, gc_mask, rect, bgd_model, fgd_model, iterations, cv2.GC_INIT_WITH_RECT)
    binary = np.where((gc_mask == cv2.GC_FGD) | (gc_mask == cv2.GC_PR_FGD), 255, 0).astype(np.uint8)
    binary = cv2.morphologyEx(binary, cv2.MORPH_OPEN, np.ones((5, 5), np.uint8))
    binary = cv2.morphologyEx(binary, cv2.MORPH_CLOSE, np.ones((11, 11), np.uint8))
    contours, _ = cv2.findContours(binary, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    largest = max(contours, key=cv2.contourArea)
    solid = np.zeros_like(binary)
    cv2.drawContours(solid, [largest], -1, 255, thickness=cv2.FILLED)
    return (solid > 0).astype(np.float64)

def composite_masked(base, overlay, overlay_mask, top, left):
    # Paste overlay (and its own mask) onto a copy of base at (top, left). Returns the composited image
    # the same size as base, for blending against the unmodified base with an irregular (non-seam) mask
    canvas = base.copy()
    mask_full = np.zeros(base.shape[:2])
    h, w = overlay.shape[:2]
    region = canvas[top:top + h, left:left + w]
    weight = overlay_mask[..., np.newaxis]
    region[:] = overlay * weight + region * (1 - weight)
    mask_full[top:top + h, left:left + w] = overlay_mask
    return canvas, mask_full
```

The cutout is resized and pasted where the dog sits, large enough to cover its hat, ears and snout entirely, so no trace of the dog is left once the mask is applied. The composite and the mask both use the same size as the fire scene, and the mask is fed through `gaussian_stack` exactly as the vertical-seam mask was in Part 2.3, so the two blend with the same code.



Figure 34a. The scene the object is pasted into.



Figure 34b. The photo the object is cut from.



Figure 35a. The cutout (masked input).



Figure 35b. The irregular mask.



Figure 35c. The blended result.

Figure 36 shows the favorite result's Laplacian stack, in the same layout as Figure 3.42 in Part 2.3: Spongebob and the fire scene weighted by the mask at three levels, and their sum. At level 0 the mask is almost as sharp as the cutout itself, so the two columns barely overlap. At level 4 the mask has spread into a soft blob, so the two columns blend into each other rather than meeting at a hard edge, which is what removes the sticker look from the final sum.

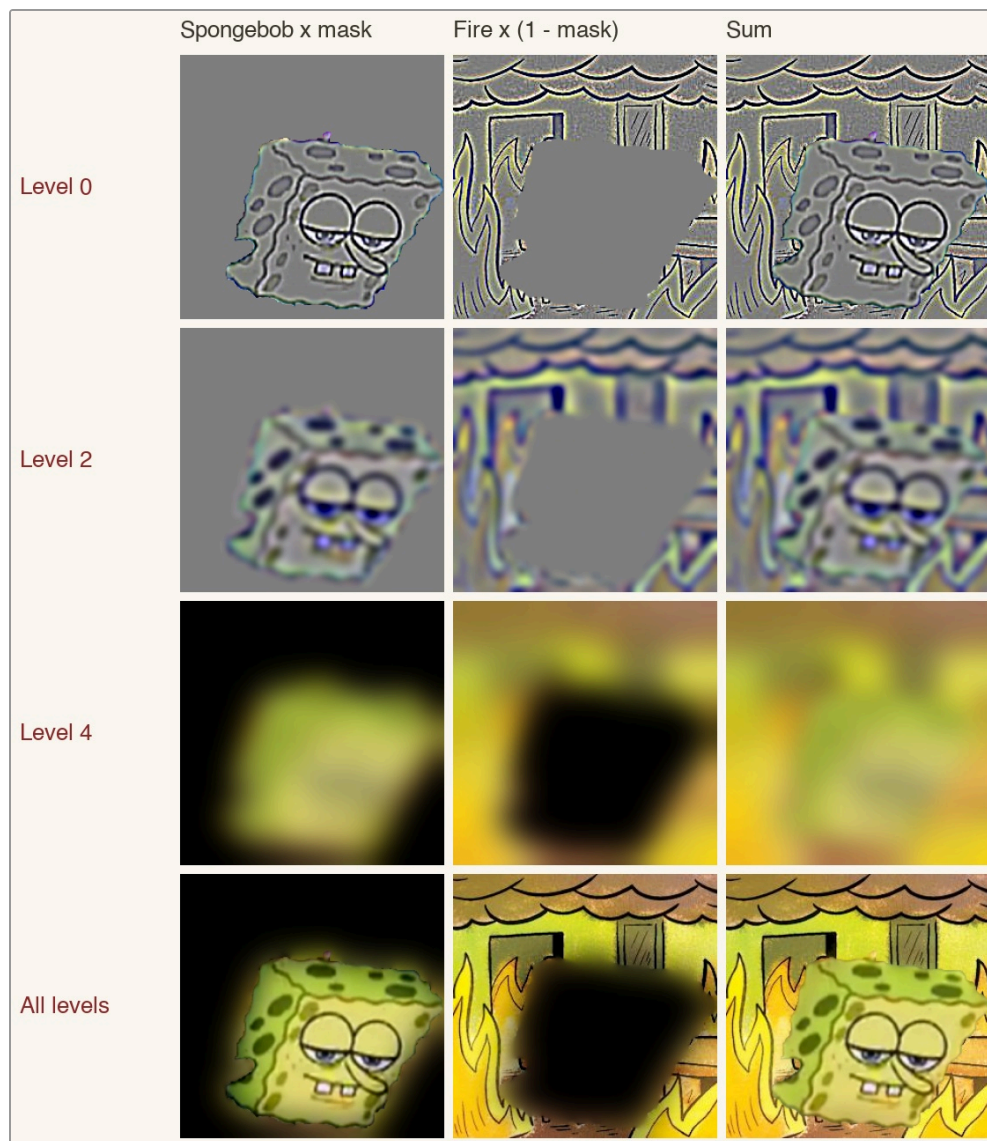


Figure 36. Spongebob and the fire scene weighted by the mask at three levels, and the total over all levels.

Takeaways

A single operation, convolution with a kernel, runs through every part of this project. The same primitive that pulls out edges in Part 1 also removes and restores frequencies for sharpening in Part 2.1, separates high and low frequencies for hybrid images in Part 2.2, and, applied repeatedly at doubling widths, builds the stack of frequency bands that Parts 2.3 and 2.4 blend. Convolution turns out to be a general-purpose tool: swapping only the kernel and how its outputs get combined is enough to move between very different-looking results, from edge detection to multiresolution blending, without changing the underlying operation at all.